SOFTWARE REQUIREMENTS SPECIFICATION

NexusChat — Real-Time Community and Collaboration Platform

Version 1.0

PES UNIVERSITY, BENGALURU

Department of Computer Science and Engineering

UE24CS341A — Software Engineering

Academic Year 2026

Revision History

Version	Date	Author	Reason for Change
1.0	2026-09-01	Project Team	Initial project baseline; review and update as requirements are agreed.

Document note: This is a working SRS draft. Replace the member placeholders and confirm requirement details with the approved project documents before final submission.

1. Introduction

1.1 Purpose

This document describes what NexusChat is expected to do and the conditions it must meet. It records the main functional requirements, quality expectations, interfaces, and security rules so that the team can build and test the application against a shared understanding.

1.2 Intended Audience and Reading Suggestions

The document is intended for the student development team, testers, project evaluator, project coordinator or product owner, and anyone who may maintain the application later. Readers looking for the product's purpose should start with Sections 1 and 2. Developers should also use Sections 3 and 5, while testers should refer to the requirements alongside the validation plan and traceability matrix.

1.3 Product Scope

NexusChat is a browser-based platform where people can join communities, take part in text channels, and exchange direct messages. The application will support user profiles, online presence, typing indicators, message editing and deletion, reactions, unread counts, notifications, message search, and permission-based moderation. Community and channel actions will be controlled by roles and permissions.

The MVP focuses on text-based communication. Voice and video calls, screen sharing, native mobile applications, and end-to-end encryption are outside the currently approved scope unless the team and evaluator formally agree to change it.

1.4 References

- PES University, Department of Computer Science and Engineering, UE24CS341A Software Engineering Project Guidelines, 2026.
- NexusChat project synopsis, approved SRS, architecture and design specification, validation specification, test plan, field layouts, and requirement traceability matrix.

The team should check the project package for the current versions of these references and update this list if the approved document set changes.

2. Overall Description

2.1 Product Perspective

NexusChat is being developed as a new web application. The React frontend will run in a browser and communicate with a Node.js backend. Standard request-and-response operations will use a REST API over HTTPS, while WebSockets will carry real-time events such as new messages, presence changes, and typing indicators. PostgreSQL will store persistent

application data. Docker will support a repeatable development and deployment environment, and Jenkins will be used for the project's CI/CD workflow.

2.2 Product Functions

- Register accounts, sign in and sign out, and manage authenticated sessions.
- View and update permitted profile information.
- Create communities, manage membership, and handle invitations.
- Create, rename, and delete text channels according to permissions.
- Send and receive channel messages in real time.
- Exchange one-to-one direct messages.
- Show online, idle, and offline presence, along with typing indicators.
- Edit or delete messages when authorized, and add or remove reactions.
- Apply community roles and permissions and carry out moderation actions.
- Show unread counts and relevant notifications, and search accessible messages.
- Record security-relevant moderation actions in an audit trail.
- Provide a responsive browser interface with understandable validation and error messages.

2.3 User Classes and Characteristics

User Class	Typical Needs	Expected Access
Guest	Understand the service and create or access an account.	Registration and login screens only.
Member	Join communities and communicate with other members.	Profile, communities, channels, messages, direct messages, reactions, search, and personal notification views.
Moderator	Help maintain a community and respond to rule violations.	Member capabilities plus moderation actions allowed by the assigned permissions.
Community Admin	Manage a community and its structure.	Community, membership, channel, role, and permission management, subject to system rules.
System Maintainer	Operate, troubleshoot, and deploy the application.	Operational and deployment access as defined by the project; this does not automatically grant access to private user conversations.

2.4 Operating Environment

- A modern version of a supported browser, such as Chrome/Chromium, Firefox, or Safari.
- Node.js LTS for the backend runtime.
- PostgreSQL for persistent relational data.
- HTTPS for web API traffic and WSS for WebSocket traffic in production.
- Docker for consistent environments and Jenkins for CI/CD.

2.5 Design and Implementation Constraints

- The project will follow the Agile process required by the course and will track work in the team's GitHub repository and backlog.
- Changes to the protected main branch will be submitted through pull requests and reviewed before merging.
- Automated build and test checks, code review, and security validation are part of the project workflow.
- WebSockets will be used for real-time messaging and related live events.

- The approved technology stack is React, Node.js LTS, PostgreSQL, WebSockets, Docker, Git/GitHub, Jenkins, and an agreed testing and code-quality toolchain.
- Each team member should own a defined area of functionality and contribute to implementation as well as documentation.

2.6 Assumptions and Dependencies

- Users have a working network connection and a supported browser.
- The application can access the configured PostgreSQL database and required WebSocket services.
- The team has access to the GitHub repository, project backlog, CI/CD environment, and test tools.
- The deployment environment can provide HTTPS/WSS and store secrets outside source control.
- Voice/video calling and other features explicitly outside the MVP will not be added without approval.

3. External Interface Requirements

3.1 User Interfaces

The interface should make the main chat workflow easy to follow. At minimum, the browser application will include:

- Registration and login screens.
- A community navigation area and channel list.
- A message timeline and message composer.
- A direct-message view.
- A member list showing presence where permitted.
- Community and channel administration controls for authorized users.
- Search and notification views.
- Clear feedback for invalid input, failed requests, permission errors, and connection problems.

The layout should adapt to common desktop and laptop screen sizes. Any responsive behaviour beyond the agreed MVP should be confirmed during design and validation.

3.2 Software Interfaces

Interface	Purpose	Expected Data / Notes
REST API	Account, profile, community, channel, search, and moderation operations.	JSON request and response bodies; errors should use a consistent format.
WebSocket API	Real-time messages, presence, typing, and reaction events.	Versioned JSON event envelope with event type, timestamp, and payload.
PostgreSQL	Persistent application state.	Users, communities, memberships, channels, messages, roles, reactions, notifications, and audit records as finalized in the data model.
Jenkins	Continuous integration and delivery workflow.	Build, test, and configured analysis or deployment stages.
SonarQube or equivalent	Code-quality and static-analysis feedback.	Quality and security findings where the selected tool supports them.
Jest/Supertest or equivalent	Automated testing.	Unit and integration tests for relevant backend, API, and application flows.

3.3 Communications Interfaces

The deployed application shall use HTTPS for browser-to-server request/response traffic and WSS for WebSocket traffic. WebSocket messages should follow a documented, versioned JSON event format. The server should validate incoming events and return an understandable error or reject an event when it is malformed or the sender is not authorized.

4. Analysis Models

The principal actors are Guest, Member, Moderator, and Community Admin. The System Maintainer supports deployment and operations. The detailed use-case diagram, use-case descriptions, and data model should be maintained in the Architecture and Design Specification so the same model is not duplicated inconsistently across documents.

The use-case model should cover account access, profile updates, community membership, channel management, channel messaging, direct messaging, message edits and deletion, reactions, search, notifications, and moderation. Each use case should identify its actor, preconditions, main flow, alternative or error flows, and postconditions.

Insert or reference the approved UML use-case diagram here once it has been reviewed by the team.

5. System Features and Functional Requirements

The requirements below describe the expected behaviour of the MVP. “Shall” indicates a required behaviour. The team should keep the IDs stable so they can be referenced in test cases and the Requirement Traceability Matrix.

5.1 Identity and Profile

FR-AUTH-01 [High]: The system shall allow a guest to register using a unique username or email address and a password. If the username or email is already registered, the system shall reject the registration and explain which field needs attention without exposing another user’s private information.

FR-AUTH-02 [High]: The system shall authenticate a user only when the submitted credentials are valid. Invalid credentials shall not create an authenticated session, and the response shall not reveal whether a particular account exists.

FR-AUTH-03 [High]: The system shall allow an authenticated user to log out and shall invalidate the associated active session or token according to the chosen authentication design.

FR-AUTH-04 [Medium]: The system shall allow a user to view and update the profile fields permitted by the application. Invalid values shall be rejected with a clear validation message.

5.2 Community and Channel Management

FR-COM-01 [High]: The system shall allow an authenticated member to create a community with a name that is unique for that owner. If the name does not meet the agreed validation rules or conflicts with an existing community owned by that user, creation shall fail with a useful message.

FR-COM-02 [High]: The system shall allow a community admin with the required permission to invite users and manage community membership. Unauthorized users shall not be able to issue invitations or change membership.

FR-COM-03 [High]: The system shall allow an authorized community admin to create, rename, and delete text channels. The system shall reject these actions when the requester lacks permission or the supplied channel details are invalid.

FR-COM-04 [High]: The system shall check the requester’s role and channel/community permissions on the server before carrying out any protected action. A failed permission check shall leave the protected resource unchanged.

5.3 Real-Time Messaging and Direct Messages

FR-MSG-01 [High]: The system shall allow a member to send a text message to a channel that the member is permitted to access. Empty or invalid messages shall be rejected according to the agreed message-validation rules.

FR-MSG-02 [High]: When a valid channel message is accepted, the system shall store it and deliver a corresponding event through WebSockets to connected clients authorized to view that channel. A client that is disconnected shall be able to retrieve persisted messages when it reconnects and opens the channel.

FR-MSG-03 [High]: The system shall allow two eligible users to exchange one-to-one direct messages. Users who are not participants in a direct conversation shall not be able to retrieve its messages or receive its real-time events.

FR-MSG-04 [Medium]: The system shall allow a message author to edit or delete their own message, subject to the agreed rules. A moderator or admin may act on another user's message only when the relevant permission allows it. Unauthorized changes shall be rejected.

FR-MSG-05 [Medium]: The system shall publish typing and presence events only to users authorized to see the relevant conversation or community. If a connection is lost, the system shall update presence according to the agreed timeout and reconnection rules.

5.4 Reactions, Search, and Notifications

FR-UX-01 [Medium]: The system shall allow a member to add or remove a reaction on a message they can access. The system shall reject reaction changes for inaccessible messages or invalid reaction values.

FR-UX-02 [Medium]: The system shall allow a member to search messages only within communities and conversations they are authorized to access. Search results shall not reveal message text or metadata from inaccessible locations.

FR-UX-03 [Medium]: The system shall maintain unread counts and notify users about relevant new messages. Reading a conversation shall update its unread state according to the agreed read-tracking rules.

5.5 Moderation and Audit

FR-MOD-01 [High]: A moderator or admin shall be able to remove a message only in a community where they have the required permission. The system shall reject unauthorized attempts and shall record the moderation action in the audit trail.

FR-MOD-02 [High]: A moderator or admin shall be able to restrict or remove a member only when their role permits that action. The system shall validate the target and permission before applying the change.

FR-MOD-03 [High]: The system shall record security-relevant moderation events, including the actor, action, affected resource, and timestamp. Audit records shall not contain passwords, authentication tokens, or other secrets.

6. Nonfunctional Requirements

6.1 Performance Requirements

The following are project targets and must be checked under a documented test load. The team should record the dataset, environment, number of concurrent clients, and measurement method in the test plan.

- The target for WebSocket message acknowledgement is no more than 500 ms at the 95th percentile (p95) under the defined test load.
- Core UI/API operations should complete within 2 seconds under normal test conditions.
- Search should return results within 2 seconds for the defined evaluation dataset.

6.2 Safety Requirements

NexusChat is not a safety-critical system. However, actions that can remove a channel, change community access, or otherwise affect multiple users should require the appropriate permission and a confirmation step where the action is destructive. The system should explain when an action cannot be completed rather than silently failing.

6.3 Security Requirements

- Passwords shall be stored using an appropriate password-hashing method and shall never be stored in plaintext.
- Protected API endpoints and WebSocket events shall require authentication and authorization as appropriate.
- Permission checks shall be enforced on the server; hiding a button in the frontend is not sufficient.
- User input shall be validated, and message content shall be rendered safely to reduce injection and cross-site scripting risks.
- Rate limiting or equivalent abuse controls shall be applied to sensitive or easily abused operations.
- Security-relevant moderation events shall be recorded in the audit trail.
- Secrets and environment-specific configuration shall be supplied through environment or secret-management configuration, not committed to the repository.
- Logs and error messages shall not expose passwords, tokens, or other sensitive secrets.

6.4 Software Quality Attributes

Attribute	Expected Approach / Target
Maintainability	Keep the React UI, backend API, WebSocket handling, and persistence responsibilities separated into understandable modules.
Testability	Provide unit tests for core logic and integration tests for API, database, and WebSocket flows.
Reliability	Handle disconnects and reconnects gracefully and avoid creating duplicate or corrupted messages during retries.
Usability	Make common chat actions understandable without requiring users to read technical documentation. Provide clear validation and error feedback.
Portability	Use Docker and documented configuration so the project can be started consistently in the intended environments.
Security	Resolve critical security findings before release, or document and obtain approval for any accepted exception.

6.5 Business and Domain Rules

- Only members of a private community may access its private content.
- Only users with the required administrative permission may change community roles and protected channel permissions.
- Moderators may act only in communities where they have been granted the relevant permission.
- Users may edit or delete their own messages; actions on other users' messages require an applicable moderation permission.
- Deleted messages shall no longer appear in normal user-facing views. Any retention or audit requirements for deleted content must follow the approved project rules.

7. Other Requirements

- The database shall use primary keys, foreign keys, and uniqueness constraints where needed to protect data integrity.

- Dates and times shall be stored consistently, preferably in UTC, and displayed to users in a suitable local format.
- Application logs shall support troubleshooting without recording passwords, authentication tokens, or other secrets.
- Field-level validation rules and data formats shall be kept consistent with the approved Field Layouts spreadsheet.
- Requirement IDs and test references shall remain consistent with the Requirement Traceability Matrix.

Appendix A: Glossary

Term	Meaning in NexusChat
Community / Server	A persistent group space with members, roles, and channels.
Channel	A text conversation space within a community.
DM (Direct Message)	A private one-to-one conversation between two users.
WebSocket	A persistent, two-way connection used to exchange real-time events.
Presence	A user availability state, such as online, idle, or offline.
Role	A named set of permissions assigned within a community.
MVP (Minimum Viable Product)	The smallest agreed version of the product that meets the project's core goals.
RTM (Requirement Traceability Matrix)	A record linking requirements to design elements, implementation, and tests.
CI/CD	Continuous integration and continuous delivery/deployment practices used to automate build, test, and release steps.

Appendix B: Field Layouts

Use 08_Field_Layouts.xlsx as the detailed reference for field names, formats, validation rules, and required/optional status. Keep this SRS and the spreadsheet aligned when field rules change.

Appendix C: Requirement Traceability Matrix

Use 07_Requirement_Traceability_Matrix.xlsx as the initial RTM. Each functional and nonfunctional requirement should be mapped to relevant design components and test cases. Update the matrix whenever a requirement is changed, added, deferred, or removed.